TRIBHUVAN UNIVERSITY
FACULTY OF
COMPUTER SCIENCE
AND
INFORMATION TECHNOLOGY

Proposal on
Plagiarism Checker for NCCS

Submitted to:
National College of Computer Studies (NCCS)
Paknajol, Kathmandu

Submitted by:
Sabhyata Aryal
College RegNo: NCCS CSIT 519

Contents

Chapter 1: Introduction 1
Chapter 2: Problem Statement 2
Chapter 3: Objectives 3
General Objective 3
Specific Objectives 3
Chapter 4: Methodology 4
4.1 Requirement Identification 4
4.1.1 Background Study 4
4.1.2 Literature Review 5
4.2 Requirement Analysis 6
4.2.1 Functional Requirements: 6
4.2.2 Non-functional Requirements 6
4.3 Feasibility Analysis 6
Chapter 5: System Design 7
5.1 Overview of System Architecture 7
5.2 User Roles 8
5.3 Use Case Diagram 8
5.4 System Flowchart 9
5.5 Project Schedule / Gantt Chart 10
Chapter 6: Algorithms Used 11
6.1 N-Gram Based Similarity Detection 11
6.2 TF-IDF with Cosine Similarity 11
Chapter 7: Expected Outcome 12
Chapter 8: Future Enhancements 12
Chapter 9: Conclusion 13
Chapter 10: References 14

List of Figures

Figure 1 Use case diagram 8
Figure 2 System Flowchart 9
Figure 3 Gantt Chart 10

Chapter 1: Introduction

Academic integrity is a fundamental principle of higher education. [1] Originality in student work reflects genuine learning, critical thinking, and research ability. In academic institutions such as NCCS, final year project reports are an essential requirement for the completion of undergraduate programs and are expected to demonstrate independent effort and innovation.

However, with increasing access to digital content and repeated submission of similar projects across different academic batches, the risk of plagiarism has significantly increased. [2]Manual checking of project reports is time-consuming, error-prone, and impractical due to the large volume of documents involved. Existing commercial plagiarism detection tools are often expensive and may not be customized to the specific requirements of the institution. [3]

To address these issues, this project proposes the development of an internal plagiarism detection system for NCCS. The system will maintain a centralized repository of previously submitted final project reports and will allow teachers to upload new reports for automated textual similarity checking. The system aims to assist teachers in identifying similarities between documents efficiently.

Chapter 2: Problem Statement

At present, NCCS lacks an automated system to verify the originality of final year project reports submitted by students. Teachers must rely on manual inspection, which is inefficient and unreliable, especially when dealing with a large number of reports accumulated over several academic years.

The absence of a centralized digital repository of previously submitted project reports makes it difficult to detect repeated content across batches. As a result, there is a risk that students may submit reports with significant copied content from earlier submissions without being detected.

Therefore, there is a need for an automated plagiarism detection system that can compare newly submitted reports against a database of previously submitted NCCS project reports and generate a detailed similarity report to support academic integrity.

Chapter 3: Objectives

General Objective

To develop an automated plagiarism detection system that checks textual similarity between newly submitted student project reports and previously submitted NCCS final project reports.

Specific Objectives

- To create a centralized repository of past final year project reports
- To extract and preprocess text from uploaded project documents
- To detect textual similarity using efficient text comparison algorithms
- To generate similarity percentages and identify matched content
- To assist teachers in evaluating the originality of student reports

Chapter 4: Methodology

4.1 Requirement Identification

4.1.1 Background Study

Plagiarism has become a growing concern in academic institutions due to the increased availability of digital documents and ease of copying electronic content. In higher education, final year project reports play a crucial role in evaluating a student's understanding of research, problem-solving, and technical skills. Ensuring the originality of these reports is essential for maintaining academic standards and institutional credibility. [1]

Traditionally, plagiarism detection in many colleges, including NCCS, is performed manually by supervisors or evaluation committees. This approach relies heavily on human memory and visual inspection, which is inefficient and unreliable when dealing with a large number of documents submitted over multiple academic years. As the volume of archived project reports increases, manual comparison becomes practically impossible. [4]

Commercial plagiarism detection tools such as Turnitin and iThenticate are widely used internationally. However, these tools are often costly, depend on external databases, and may not allow institutions to maintain full control over their internal academic documents. Additionally, internet-based plagiarism checkers may not effectively detect similarities between previously submitted institutional reports if those documents are not publicly available. [2]

The background study highlights the need for a localized plagiarism detection system that focuses specifically on internal academic documents. By maintaining a centralized repository of past final project reports and applying automated text similarity techniques, NCCS can efficiently detect repeated content across batches while preserving data privacy and reducing dependency on third-party tools. [3]

4.1.2 Literature Review

Plagiarism detection has been widely studied in the field of Natural Language Processing (NLP) and information retrieval. Existing research categorizes plagiarism detection techniques into lexical-based, syntactic-based, semantic-based, and hybrid approaches. [2] Among these, lexical-based approaches are the most commonly used due to their simplicity, efficiency, and reliability in detecting direct text reuse.

One of the widely used lexical techniques is N-gram analysis, where documents are divided into contiguous sequences of words or characters. Research studies have shown that N-gram–based methods are highly effective in detecting copy-paste plagiarism and minor text modifications. The similarity between documents is often measured using Jaccard similarity, which calculates the ratio of shared N-grams between two documents. This approach is computationally efficient and suitable for large document repositories.

Another extensively studied technique is Term Frequency–Inverse Document Frequency (TF-IDF) combined with cosine similarity. TF-IDF assigns weights to terms based on their importance within a document and across a document collection. Studies indicate that TF-IDF is effective in reducing the influence of common words while emphasizing meaningful terms. Cosine similarity is then used to measure the similarity between document vectors. This method performs better than simple keyword matching and is capable of detecting partial plagiarism and limited paraphrasing. [5]

Several researchers suggest combining multiple similarity detection techniques to improve accuracy. Hybrid systems that integrate N-gram matching with TF-IDF-based similarity have demonstrated better performance than single-method approaches. Such systems balance efficiency and accuracy, making them suitable for academic plagiarism detection where both speed and reliability are required. [6]

More advanced approaches, such as semantic similarity detection using word embeddings and deep learning models, have also been explored in recent studies. While these methods can detect idea-level plagiarism, they require significant computational resources and large training datasets. As a result, many researchers conclude that traditional text similarity techniques remain the most practical solution for institution-level plagiarism checkers.

4.2 Requirement Analysis

4.2.1 Functional Requirements:

- Teachers can upload previously submitted final project reports
- Teachers can upload new student project reports
- The system extracts text from PDF and DOCX files
- The system preprocesses the extracted text
- The system compares the new report with stored reports
- The system generates a plagiarism (similarity) report

4.2.2 Non-functional Requirements

- The system should provide accurate similarity detection
- The system should respond within reasonable time
- The system should ensure data security and access control
- The system should be easy to use for teachers

4.3 Feasibility Analysis

a. Economic Feasibility

The project uses open-source technologies and does not require paid software licenses, making it cost-effective.

b. Technical Feasibility

The system will be developed using Python and open-source NLP libraries, which are well-supported and widely used. Required tools and technologies are readily available.

c. Operational Feasibility

Teachers can easily operate the system through a simple web interface without requiring specialized technical skills.

d. Schedule Feasibility

The project can be completed within the academic timeline by following a structured development plan.

Chapter 5: System Design

5.1 Overview of System Architecture

The system follows a web-based architecture consisting of a user interface for teachers, a processing module for text extraction and similarity detection, and a database to store project reports and processed text.

1. **User Interface (UI)**
    - Built as a web-based interface using HTML, CSS, and a Python web framework
    - Allows teachers to upload past final project reports and new student reports
    - Displays plagiarism checking results including similarity percentage and matched source documents
2. **Document Processing Module**
    - Responsible for handling uploaded PDF and DOCX files
    - Extracts textual content from documents
    - Performs text preprocessing such as lowercasing, removal of stop words, punctuation, and whitespace normalization
3. **Similarity Detection Module**
    - Implements N-gram–based similarity detection using Jaccard similarity
    - Applies TF-IDF with cosine similarity for partial plagiarism detection
    - Computes similarity scores between the newly submitted report and stored reports in the repository
4. **Plagiarism Analysis Module**
    - Aggregates similarity results from multiple algorithms
    - Identifies the highest similarity match among stored reports
    - Classifies similarity levels based on predefined thresholds
    - Prepares data required for plagiarism report generation
5. **Controller Module**
    - Coordinates interaction between the user interface, document processing, similarity detection, and database modules
    - Handles file validation, workflow control, and error handling

- Manages the overall plagiarism checking process from upload to report generation

6. **Database Module**

- Stores previously submitted project reports and extracted text
- Maintains document metadata such as document ID and upload date
- Provides stored data for similarity comparison

5.2 User Roles

- **Teacher (Admin):** Uploads reports and views plagiarism results
- **System:** Processes documents and performs similarity detection

5.3 Use Case Diagram

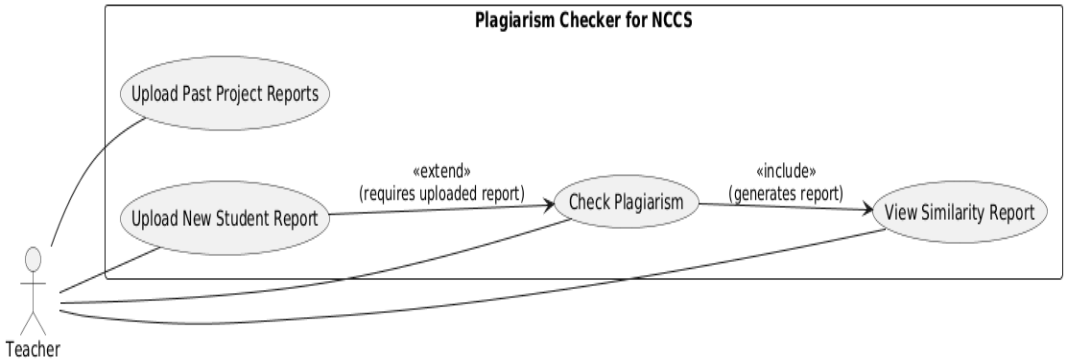

Figure 1 Use case diagram

5.4 System Flowchart

The system flowchart provides a visual representation of the sequence of operations in the system:

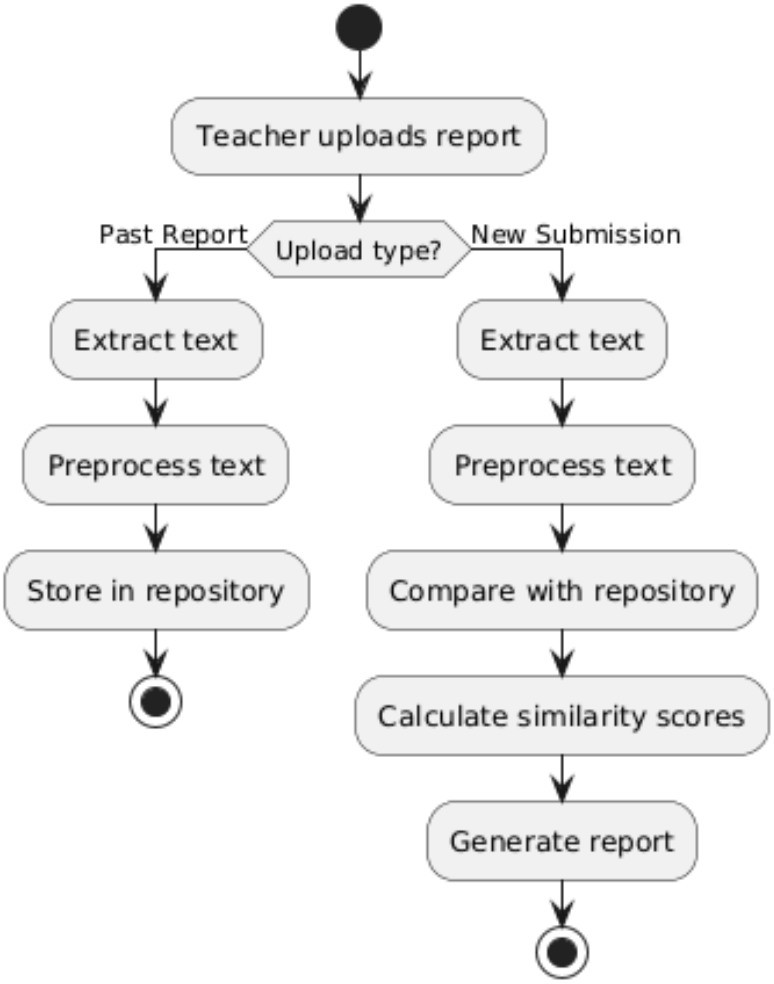

Figure 2 System Flowchart

5.5 Project Schedule / Gantt Chart

The project is divided into seven main phases, each phase is scheduled to be completed over a 3-month period, assuming 3–4 hours of work daily.

	1st week	2nd week	3rd week	4th week	5th week	6th week	7th week
Planning							
Analysis							
Design							
Coding							
Testing							
Implementation							
Documentation							

Figure 3 Gantt Chart

Chapter 6: Algorithms Used

6.1N-Gram Based Similarity Detection

N-gram analysis involves dividing text into overlapping sequences of words of length *k*. These sequences are compared between documents to detect copied text. [5]

a) Text Shingling

Documents are divided into k-word sequences to represent their structure.

b) Jaccard Similarity

Jaccard similarity measures the overlap between two sets of N-grams and is calculated as the ratio of the intersection to the union of the sets.

c) Working of the Algorithm

The N-grams of a new report are compared with the N-grams of stored reports to compute similarity scores.

d) Why N-Gram Is Suitable

It is efficient, easy to implement, and effective in detecting copy-paste plagiarism.

6.2 TF-IDF with Cosine Similarity

TF-IDF assigns weights to terms based on their importance in a document. Cosine similarity measures the angle between document vectors to determine similarity. [6] This method helps detect partial plagiarism and paraphrased content.

Chapter 7: Expected Outcome

The expected outcome of this project is a functional plagiarism checker for NCCS that:

- Detects textual similarity accurately
- Generates clear similarity reports
- Reduces manual workload for teachers
- Enhances academic integrity at NCCS

Chapter 8: Future Enhancements

- Integration of semantic plagiarism detection
- Internet source comparison
- Student submission portal
- Report export in PDF format

Chapter 9: Conclusion

The proposed plagiarism detection system for NCCS final project reports addresses a critical need for maintaining academic integrity and ensuring the originality of student work. By implementing a centralized repository of previously submitted reports and integrating automated text similarity detection techniques, the system provides a structured, reliable, and efficient method for identifying potential instances of plagiarism. This approach eliminates the inefficiencies and subjectivity associated with manual checking, offering teachers a practical and scalable solution.

The system architecture has been designed with a clear separation of concerns, including user interface, document processing, similarity detection, plagiarism analysis, controller, and database modules. Each module contributes to a robust and maintainable system that can be enhanced over time. The use of N-gram based Jaccard similarity alongside TF-IDF with cosine similarity ensures a balanced approach, capable of detecting both direct copy-paste plagiarism and partial paraphrasing, thereby increasing the accuracy and credibility of the results.

Beyond its technical implementation, the system also strengthens institutional policies on academic honesty. By generating detailed similarity reports, highlighting matched content, and providing clear similarity percentages, the system equips teachers with actionable information to make informed decisions. This not only supports fair assessment practices but also encourages students to uphold ethical research standards.

Furthermore, the proposed system lays the foundation for future enhancements, such as semantic plagiarism detection, integration with internet-based sources, and the inclusion of a student submission portal. These potential upgrades would further expand the system's capabilities, making it a comprehensive academic integrity solution for the institution.

In conclusion, the project combines practical relevance with academic rigor. It demonstrates the application of natural language processing techniques and algorithmic approaches in a real-world educational context. By automating plagiarism detection and maintaining a centralized document repository, the system enhances the quality of assessment, reduces manual workload, and safeguards the credibility of NCCS's academic programs. The successful implementation of this system will represent a significant step toward improving the evaluation process and promoting a culture of honesty and originality among students.

Chapter 10: References

[1] S. M. z. E. a. M. P. B. Stein, Strategies for retrieving plagiarized documents," Proceedings of the SIGIR Workshop on Plagiarism Analysis, 2007.

[2] D. S. W. a. A. A. S. Schleimer, Winnowing: Local algorithms for document fingerprinting," Proceedings of the ACM SIGMOD International Conference, 2003, pp. 76-85.

[3] A. W. a. C. S. Y. G. Salton, A vector space model for automatic indexing," Communications of the ACM, vol. 18, 1975, pp. 613-620.

[4] F. K. a. E. Z. H. Maurer, Plagiarism—A survey," Journal of Universal Computer Science,, vol. 12, 2006, pp. 1050-1084.

[5] M. Clough, Plagiarism in natural and programming languages: an overview of current tools and technologies," Research Memorandum, University of Sheffield, 2000.

[6] C.Park, Plagiarism by university students—literature and lessons," Assessment & Evaluation in Higher Education, vol. 28 , 2003, pp. 471-488.